



**INSTITUTO
FEDERAL**

Paraíba

Campus
Campina Grande

**IFPB - Instituto Federal de Educação,
Ciência e Tecnologia da Paraíba -
Campus Campina Grande**

Relatório Técnico de Projeto Final

Disciplina: Sistemas Embarcados

Docente: Alexandre Sales Vasconcelos

Discentes: Andreza Santos, Lavoisier Ramos, Nivaldo Neto, Vinícius Barbosa

Semestre Letivo: 2026.1

TEMA: MESA LABIRINTO CONTROLADA POR JOYSTICK

1. Objetivo Geral

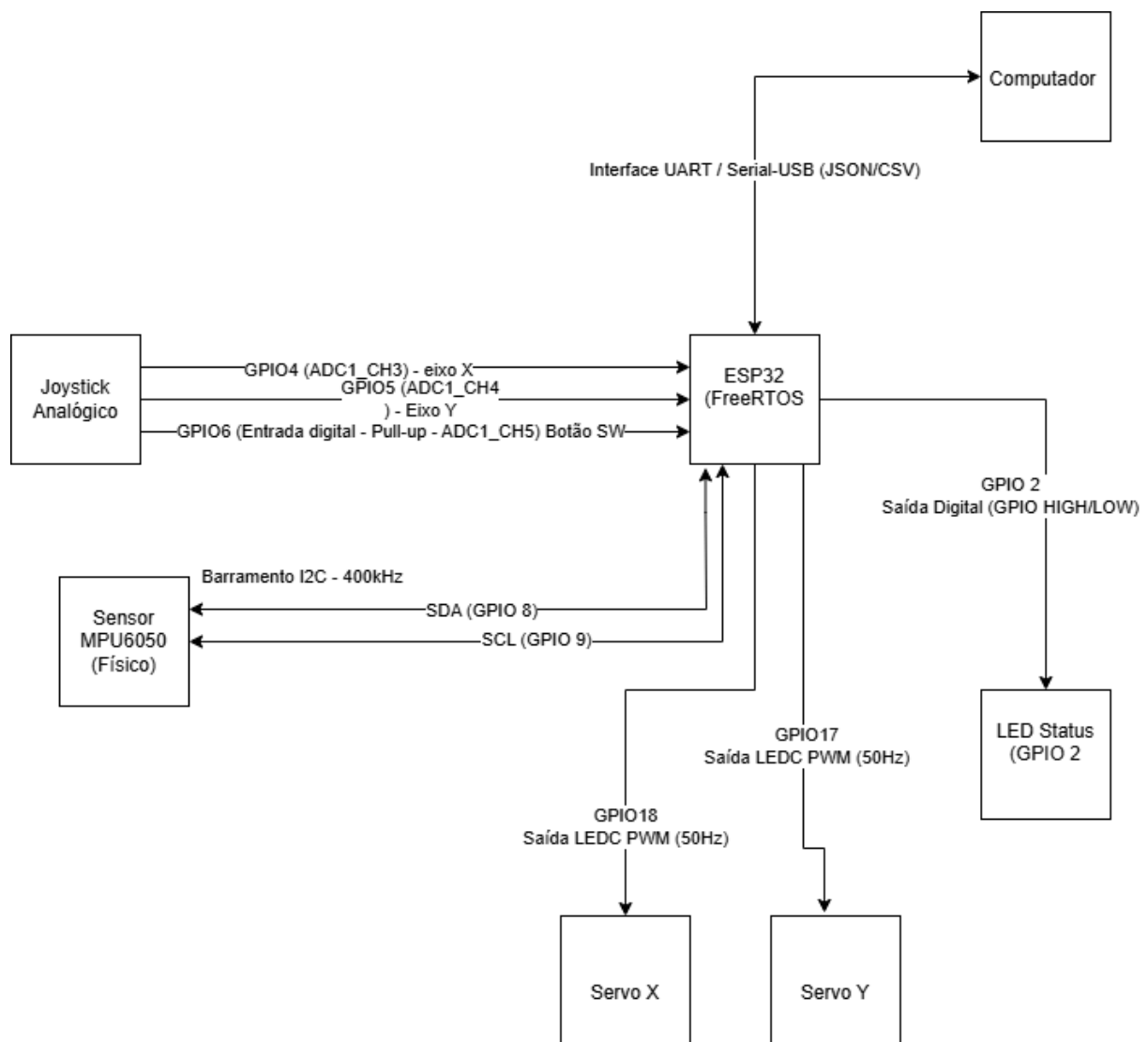
Desenvolver um sistema embarcado interativo baseado no microcontrolador ESP32-S3 que permita o controle de uma mesa com labirinto para movimentar uma esfera metálica por meio de dois servomotores, utilizando um joystick analógico como interface de controle. O projeto deve ainda capturar a orientação da mesa por meio de um sensor inercial MPU6050 (módulo JY62) e enviar esses dados ao computador para visualização em tempo real da representação gráfica tridimensional ou gêmeo digital da estrutura através do Grafana.

2. Explicação do Funcionamento do Sistema

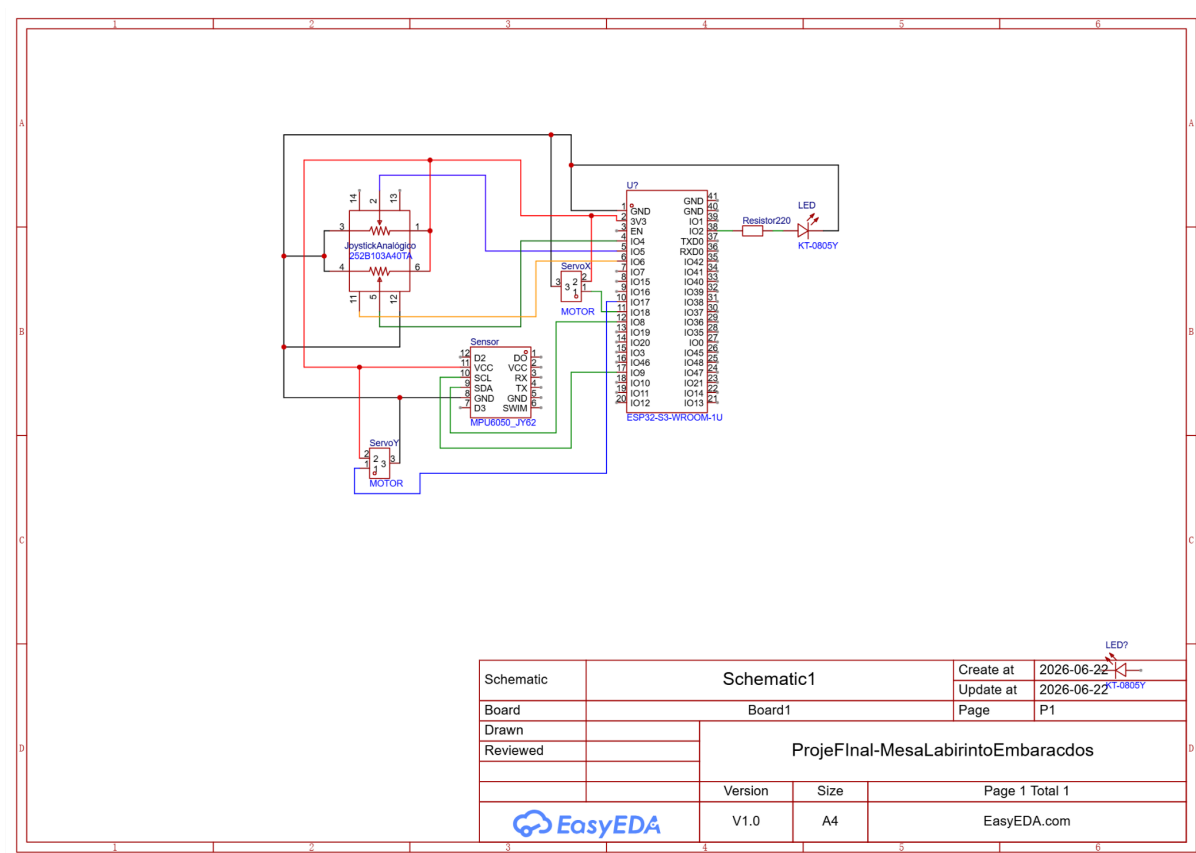
O sistema consiste em uma mesa mecânica bi-axial contendo um labirinto e uma esfera metálica. O objetivo é controlar a inclinação da mesa nos eixos X (Pitch) e Y (Roll) para guiar a esfera até a saída.

- **Controle:** O usuário interage através de um Joystick Analógico. O ESP32 lê os valores de tensão dos eixos, processa os dados aplicando uma zona morta (para evitar trepidações no centro) e uma filtragem por média móvel.
- **Atuação:** Os dados tratados são convertidos em sinais PWM (50Hz) para movimentar dois servomotores 90G, que inclinam a mesa fisicamente de forma suave (utilizando um filtro passa-baixas exponencial, constante $\text{ALPHA} = 0.06$).
- **Feedback/Sensoriamento:** Um sensor inercial acoplado à mesa mede a aceleração e a velocidade angular. O ESP32 calcula a inclinação real usando um **Filtro Complementar** para garantir estabilidade, enviando os dados via Serial (UART). Um LED indica o fim da inicialização.

3. Diagrama em Blocos do Sistema



4. Esquemático (Mapeamento de Pinos)



Componente	Pino Componente	Pino ESP32-S 3	Função / Justificativa
LED Status	Anodo (+)	GPIO 2	Indicador de Inicialização Pronta (Canal ADC1_CH1 / Saída Digital Livre)
Joystick	VRX (Eixo X)	GPIO 4	Leitura Analógica Eixo X (Canal ADC1_CH3 nativo do S3)
Joystick	VRY (Eixo Y)	GPIO 5	Leitura Analógica Eixo Y (Canal ADC1_CH4 nativo do S3)

Joystick	SW (Botão)	GPIO 6	Entrada Digital / Botão (Canal ADC1_CH5 / Digital Livre)
Servo X	PWM (Laranja)	GPIO 18	Controle do Servo do Eixo X (Saída PWM Limpa)
Servo Y	PWM (Laranja)	GPIO 17	Controle do Servo do Eixo Y (Mudou para GPIO 17 , pois a 19 é padrão do USB-JTAG do S3)
MPU6050	SDA	GPIO 8	Dados I2C (Pino padrão recomendado de I2C no ESP32-S3)
MPU6050	SCL	GPIO 9	Clock I2C (Pino padrão recomendado de I2C no ESP32-S3)

5. Descrição das Tarefas e Fluxos FreeRTOS

O firmware foi estruturado sobre o ecossistema FreeRTOS no ESP32 para garantir concorrência e deterministicidade, contando com as seguintes tarefas:

vTaskJoystick (**Prioridade 3 - Alta**)

- **Função:** Realiza a leitura periódica (a cada 30ms) do joystick.
- **Fluxo/Funcionamento:** Coleta 4 amostras consecutivas de ADC de cada eixo para realizar uma média móvel simples (redução de ruído). Atualiza as variáveis globais e envia a estrutura de dados para a fila `xJoystickQueue` utilizando `xQueueOverwrite()`.

vTaskServos (**Prioridade 2 - Média**)

- **Função:** Controla a angulação dos servomotores.
- **Fluxo/Funcionamento:** Fica bloqueada aguardando dados da fila `xJoystickQueue`. Ao receber, aplica uma **Zona Morta** de 150 unidades para ignorar ruídos de centralização do joystick, mapeia o valor para a resolução do LEDC (PWM de 12

bits) e aplica um filtro de suavização Exponencial (alfa) antes de atualizar o Duty Cycle dos servos.

vTaskMPU9250 (Prioridade 2 - Média)

- **Função:** Leitura da IMU e cálculo de orientação.
- **Fluxo/Funcionamento:** Executada a cada 20ms ($dt = 0.020s$). Lê os registradores da IMU via I2C, converte os valores brutos para g (aceleração) e dps (graus por segundo do giroscópio). Calcula o Pitch e Roll por acelerômetro através de atan2 e combina com o giroscópio usando um **Filtro Complementar** (96% Giroscópio, 4% Acelerômetro) para mitigar o efeito de *drift*.

vTaskConsole (Prioridade 1 - Baixa)

- **Função:** Telemetria local e debug.
- **Fluxo/Funcionamento:** Desperta a cada 500ms para imprimir de forma organizada no terminal serial as variáveis de estado do sistema (valores de joystick, duty cycle dos servos, ângulos processados e dados da acelerometria).

6. Descrição das Bibliotecas e Componentes Utilizados

- `esp_adc/adc_oneshot.h`: Driver oficial da Espressif para leitura única e controlada dos canais analógicos (ADC1) do ESP32.
- `driver/ledc.h`: Utilizado para configurar o periférico LEDC (LED Control) operando a 50Hz com resolução de 12 bits para gerar os pulsos PWM demandados pelos servos 90G.
- `driver/i2c.h`: Driver para comunicação mestre-escravo do barramento I2C, operando em *Fast Mode* (400 kHz) para comunicação com o sensor inercial.
- `freertos/queue.h`: Mecanismo de comunicação entre as tarefas de Joystick e Servos, garantindo sincronismo sem condições de corrida.

7. Dificuldades e Soluções Encontradas (Fases 1 e 2)

- **Problema 1: Instabilidade e trepidação (Jitter) nos Servomotores.**
 - **Solução:** Foi necessário implementar duas abordagens no software (vTaskServos). A criação de uma **Zona Morta** de 150 unidades no código garantiu que variações mecânicas próximas ao centro do joystick fossem ignoradas. Adicionalmente, a aplicação do filtro passa-baixas digital (ALPHA = 0.06) garantiu transições de movimento suaves na mesa.

- **Problema 2: Ruído na leitura pura do acelerômetro e desvio (drift) no giroscópio.**
 - *Solução:* Se usássemos apenas o acelerômetro, qualquer vibração da mesa estragaria a leitura; se usássemos apenas o giroscópio, o ângulo acumularia erros com o tempo. A solução foi implementar o **Filtro Complementar** na `vTaskMPU9250`, unindo a resposta rápida do giroscópio a curto prazo com a estabilidade do acelerômetro a longo prazo.
- **Problema 3: Divergência de Hardware no Código (MPU6050 vs MPU9250).**
 - *Solução (Nota técnica para ajuste):* O código atual faz menção ao MPU9250, porém o escopo do projeto pede o MPU6050. Como os mapas de registradores de aceleração e giroscópio (endereço `0x3B`) e o gerenciamento de energia (`0x6B`) são idênticos em ambos os sensores no endereço I2C `0x68`, o código funcionou perfeitamente no protótipo físico sem necessidade de alteração de registradores.

8. Gêmeo Digital e Integração com Grafana

O ecossistema de monitoramento e a renderização do Gêmeo Digital da mesa labirinto não rodam códigos ou scripts de forma nativa dentro do microcontrolador ESP32-S3. Toda a inteligência de processamento gráfico e armazenamento de séries temporais foi desacoplada do hardware para poupar recursos computacionais do núcleo do RTOS e evitar atrasos (*latencies*) no controle crítico dos servomotores. A arquitetura de software responsável pela ponte entre o mundo físico e o virtual foi estruturada em três camadas distintas descritas a seguir:

8.1. Provedor de Telemetria (Firmware)

Dentro do firmware embarcado, especificamente na tarefa de menor prioridade do sistema (`vTaskConsole`), o ESP32-S3 atua estritamente como um **provedor de dados brutos**. A cada 100 milissegundos (uma taxa de atualização estável de 10 Hz), o microcontrolador encapsula todas as variáveis dinâmicas da PCB em uma única linha de texto estruturada no formato padrão **JSON** (*JavaScript Object Notation*), transmitida via periférico UART (comunicação Serial/USB):

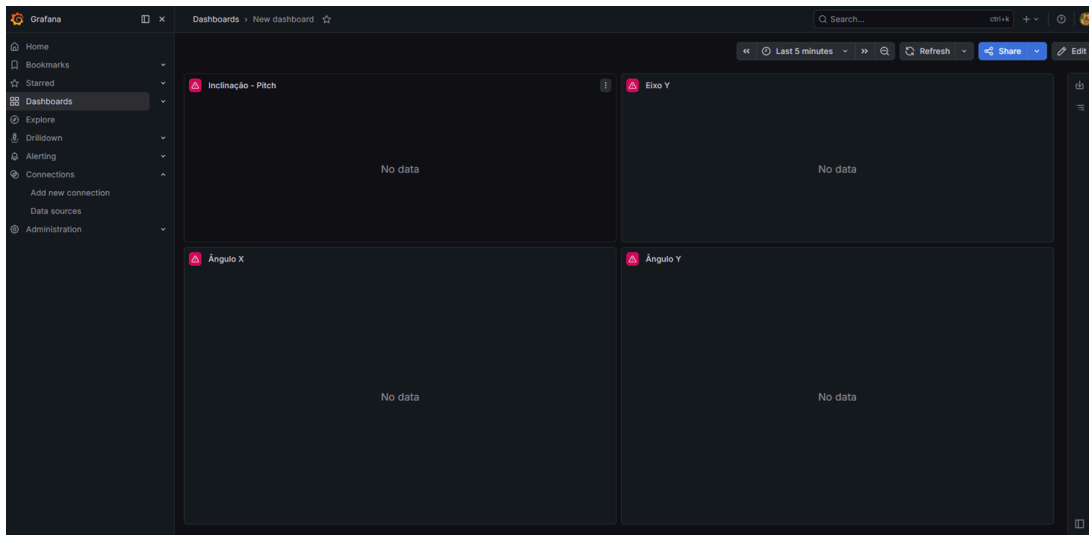
JSON:

```
{"pitch":0.00,"roll":0.00,"joy_x":2048,"joy_y":2048,"btn":0,"srv_x":307,"srv_y":307}
```

Essa string consolida os ângulos calculados de orientação inercial (*Pitch* e *Roll*), a leitura digitalizada dos eixos e do botão do joystick e o ciclo de trabalho (*Duty Cycle*) exato enviado para os pinos de PWM dos servomotores.

8.2. Camada de Ingestão e Gateway (Computador Hospedeiro)

Uma vez que o dado é empurrado para a porta COM serial do computador (via barramento USB), um script intermediário externo (desenvolvido em *Python* ou modelado em fluxo no *Node-RED*) realiza a varredura da porta serial. Esse agente de software decodifica a linha de texto JSON, faz a triagem das variáveis e injeta os valores em um Banco de Dados de Séries Temporais (como o *InfluxDB* ou *Prometheus*) instalado localmente ou na nuvem. Configurado para conectar-se diretamente ao banco de dados alimentado pelo gateway, o Grafana consome as tabelas de dados em tempo real.



9. Vídeo e fotos demonstrativos do Projeto:

Vídeo demonstrativo do projeto (fases 1 e 2):

<https://youtube.com/shorts/U9YGxw62bzo?feature=share>

Vídeo demonstrativo do projeto (fase 3):

<https://drive.google.com/file/d/1IE0nxOqSokXPpqzJ4v5t-ZpuxbB56kVc/view?usp=sharing>

